

Part 0: Set up

This notebook demonstrates the IPCA (Instrumented Principal Components Analysis) model from Kelly, Pruitt, and Su (2019). Please make sure you have installed `scipy`, `scikit-learn`, `numpy`, and `pandas`. The following command will install all required packages.

```
In [11]: # Import core libraries for numerical computation, data handling, and linear algebra.
# - numpy/pandas: array and dataframe operations
# - svds: sparse SVD, used in IPCA initialization
# - linear_model: OLS regression used in IPCA factor/Gamma estimation
# Suppress FutureWarnings from sklearn/pandas for cleaner output.
import warnings
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import time as tm
from scipy.sparse.linalg import svds
from sklearn import linear_model
warnings.simplefilter(action="ignore", category=FutureWarning)

In [13]: # Load simulated data for one replicate (n=1) of DGP variant M=1.
# Two data files are loaded:
#   c_1.csv: (N+T, P) panel of asset characteristics z_{i,t}. Each row is one
#           asset-time observation; columns are the P=50 characteristics.
#   r_1.csv: (N+T,) vector of simulated individual stock returns, stacked as
#           [r_{1,1},...,r_{1,N}], [r_{1,1},...,r_{1,N,2}, ..., r_{1,1,T}),...,r_{1,N,T}).
N, an = 1, 1
P = 50
path = "demo_data"
data = pd.read_csv(path + "/c_1d.csv" % an, delimiters=",").values
r = pd.read_csv(path + "/r1d_1d.csv" % (N, an), delimiters=",").values[:, 0]

In [14]: # Define panel dimensions and create panel identifiers.
#   N = 200 assets, T = 180 time periods, K = 5 latent factors, P = 50 characteristics.
#   'per': asset (cross-sectional) identifier for each of the N=200 stacked observations.
#   Repeats {0,1,...,N-1} for each time period.
#   'time': time period identifier for each observation.
#   Repeats each period index N times (one per asset).
#   'date': alias for 'time', used interchangeably throughout the code.
N = 200    ## Number of CS tickers
M = P      ## Number of Characteristics
T = 180    ## Number of Time Periods
K = 5      ## Number of principal components

per = np.tile(np.arange(N), T)
time = np.repeat(np.arange(T), N)
date = time

In [15]: date

Out [15]: array([ 0,  0,  0, ..., 179, 179, 179], shape=(36000,))

In [16]: # Split data into three equal time-based folds (each covering T/3 = 60 periods):
#   1. Training set (periods 0-59): used to fit model parameters.
#   2. Validation set (periods 60-119): used for early stopping / hyperparameter tuning.
#   3. Out-of-sample (periods 120-179): used to evaluate final predictive performance.

# For each split, we extract:
#   - Characteristics matrix (xtrain/xtest/xoos): input to the beta network
#   - Returns vector (ytrain/yttest/yoos): target variable
#   - Entity identifiers (trainper/testper/ooisper) and time identifiers (traindate/testdate/oosdate)
ind = range(0, int(N * T / 3))
xtrain = data[ind, :]
ytrain = r[ind]
trainper = per[ind]
traindate = time[ind]

ind = range(int(N * T / 3), int(N * (T * 2 / 3)))
xtest = data[ind, :]
ytest = r[ind]
testper = per[ind]
testdate = time[ind]

ind = range(int(N * T * 2 / 3), N * T)
xoos = data[ind, :]
yoos = r[ind]
oosdate = time[ind]
ooosper = per[ind]

In [17]: # Precompute summary statistics needed by IPCA.
#
# Nt[t]: Number of assets at time t (here always N=200, balanced panel).
#
# X (P x T): Cross-sectional "managed portfolio" moments. Column t is:
#   X[t, t] = (1/N) * Z.T * r_t
# i.e., the characteristics-weighted average return at time t.
# This is the sufficient statistic for estimating factors in IPCA.
# W (P x P x T): Cross-sectional second moment of characteristics at time t:
#   W[t, :, t] = (1/N) * Z.T * Z * Z.T
# Used by IPCA in the ALS (alternating least squares) estimation step.
L = P
udate = np.unique(time)
Nt = np.zeros(T)
for i in range(T):
    ind = time == udate[i]
    Nt[i] = np.sum(ind)
Nt = Nt.astype(int)

X = np.zeros((L, T))
for t in range(T):
    ind = time == udate[t]
    xt = r[ind].reshape(1, Nt[t]).dot(data[ind, :])
    X[:, t] = xt[0, :] / 1.0 / Nt[t]

W = np.zeros((L, L, T))
for t in range(T):
    ind = time == udate[t]
    Z = data[ind, :]
    W[:, :, t] = Z.T.dot(Z) / Nt[t]
```

Part 1: Introduction

Kelly, Pruitt, and Su (KPS, 2019)

A recent asset pricing literature has emerged challenging the "anomaly" view of characteristic-based asset return prediction. The anomaly view suggests that certain asset attributes have the power to forecast returns above and beyond the expected return variation warranted as compensation for aggregate risk exposures. Kelly, Pruitt, and Su (KPS, 2019) provide empirical evidence that these so-called anomaly asset characteristics in fact proxy for unobservable and time-varying exposures to risk factors, and shows that characteristics contain little (if any) anomalous return predictability once their explanatory power for factor exposures has been accounted for. In other words, characteristics appear to predict returns because they help pinpoint compensated aggregate risk exposures.

The asset pricing model proposed by KPS assumes that individual returns $r_{i,t}$ possess a K -factor structure:

$$r_{i,t} = \beta(z_{i,t-1})'f_t + u_{i,t} \tag{1}$$

The factors f_t are treated as latent, and the $K \times 1$ conditional factor exposure $\beta(z_{i,t-1})$ is a function of an $P \times 1$ vector of asset characteristics $z_{i,t-1}$, where P is potentially high dimensional and strictly greater than K . KPS make the simplifying assumption that the map from P characteristics to K betas is linear:

$$\beta(z_{i,t-1})' = z_{i,t-1}'\Gamma \tag{2}$$

which leads to an especially tractable estimation strategy for both the beta function and the latent factors.

There are, nonetheless, no obvious theoretical or intuitive justifications for this convenient linearity assumption. To the contrary, there are many reasons to expect that this assumption is violated. Essentially all leading theoretical asset pricing models predict nonlinearities in return dynamics as a function of state variables; Campbell and Cochrane (1999), Bansal and Yaron (2004), and He and Krishnamurthy (2013) are prominent examples. Theory also predicts complex dynamics in factor risk exposures, as shown for example in the general equilibrium model of Santos and Veronesi (2004). Moreover, Pohl et al. (2018) show that linear approximations to nonlinear models can lead to considerable errors in the model predictions for the magnitude of the equity premium or return predictability.

Part 2: IPCA

Just like the autoencoder model nests the static linear factor model, the augmented autoencoder nests the IPCA factor model proposed by KPS as a special case. KPS propose a method called IPCA to estimate their linear, time-varying beta model. IPCA solves the optimization problem:

$$\min_{\Gamma, F} \sum_{i=1}^T \sum_{t=1}^N \|r_{i,t} - z_{i,t-1}'\Gamma'f_t\|^2 = \min_{\Gamma, F} \sum_{i=1}^T \|r_i - Z_{i-1}\Gamma'f_i\|^2 \tag{17}$$

where $F = (f_1, f_2, \dots, f_T)$ and $Z_t = (z_{1,t}^T, z_{2,t}^T, \dots, z_{N,t}^T)'$, and subject to restrictions that identify a unique rotation of factors.

For comparison, consider a particularly simple version of the conditional autoencoder that uses a linear activation function and one layer of K neurons on both the beta side and the factor side of the network. In this case, $\beta_{i,t} = Z_{i-1}W_\beta'$ and $f_t = W_\Gamma x_t$, so the the estimation objective of the conditional autoencoder is:

$$\min_{W_\beta, W_\Gamma} \sum_{i=1}^T \|r_i - Z_{i-1}W_\beta'W_\Gamma x_i\|^2 \tag{18}$$

The next proposition formalizes the equivalence between IPCA and this linear conditional autoencoder.

Proposition 2. The solution to (18) is equivalent to the solution of (17) if $Z_i'Z_i = \Sigma$ for a constant matrix Σ .

In the general case where $Z_i'Z_i$ is non-constant, the two estimators are similar but no longer equivalent (as we can see from the proof). We find that the empirical performance of (17) and (18) is similar in our data.

```
In [18]: # --- Part 2: Train the IPCA Model (Kelly, Pruitt, Su 2019) ----
from IPCA import IPCA
start_time_ipca = tm.time()
ret_hat_ipca, ret_hat_ipca, Gamma_ipca = IPCA(X, W, Nt, time, udate)
end_time_ipca = tm.time()
time_ipca = end_time_ipca - start_time_ipca

In [19]: r2_ipca = np.zeros(2)
ind = range(int(N * T * 2 / 3), N * T)
r2_ipca[0] = 1 - np.mean(np.power(ret_hat_ipca[ind] - yoos, 2)) / np.mean(np.power(yoos, 2))
r2_ipca[1] = 1 - np.mean(np.power(ret_hat_ipca[ind] - yoos, 2)) / np.mean(np.power(yoos, 2))
print(f"IPCA Total R^2: {r2_ipca[0]:.4f}")
print(f"IPCA Predictive R^2: {r2_ipca[1]:.4f}")
print(f"IPCA Execution Time: {time_ipca:.2f} seconds")

IPCA Total R^2: 0.4443
IPCA Predictive R^2: 0.0372
IPCA Execution Time: 2.65 seconds
```

Part 3: Regressed-PCA

Regressed-Principal Components Analysis (RPCA)

Overview: RPCA is a computationally efficient, non-iterative alternative to iterative methods like IPCA. It compresses cross-sectional asset characteristics into managed portfolios and extracts common latent factors using standard PCA.

Core Methodology:

- Form Managed Portfolios:** Compute cross-sectional characteristic-weighted returns (X_t) for each period:

$$X_t = \frac{1}{N_t} Z_t' r_t$$

- Run PCA:** Perform PCA on the $P \times T$ matrix X to find the top K characteristic loadings (Γ') and time factors (F):

$$\min_{\Gamma, F} \sum_{i=1}^T \|X_i - \Gamma'f_i\|_2^2 \quad \text{s.t.} \quad \frac{1}{T} F F' = I_K$$

- Extract Factors:** Γ' is obtained via the top K eigenvectors of the sample covariance of X . The factor scores are computed as:

$$f_t = \Gamma^\top X_t$$

- Reconstruct Returns:** Estimate time-varying asset betas ($\hat{\beta}_{i,t} = z_{i,t-1}'\Gamma'$) to predict individual stock returns:

$$\hat{r}_{i,t} = z_{i,t-1}'\Gamma'f_t$$

Relying on a single SVD/PCA makes RPCA substantially faster and more stable than Alternating Least Squares (ALS).

- Reference:** Chen, Roussanov, and Wang (2025).

```
In [20]: from RPCA import RegressedPCA

In [21]: #ret_hat = returns using f_t at all time periods
#ret_hat = returns using expanding-window mean of f_t for out-of-sample periods

start_time_rpca = tm.time()
ret_hat_rpca, ret_hat_rpca, Gamma_rpca = RegressedPCA(X, W, Nt, time, udate)
end_time_rpca = tm.time()
time_rpca = end_time_rpca - start_time_rpca

In [22]: r2_rpca = np.zeros(2)
ind = range(int(N * T * 2 / 3), N * T)
r2_rpca[0] = 1 - np.mean(np.power(ret_hat_rpca[ind] - yoos, 2)) / np.mean(np.power(yoos, 2))
r2_rpca[1] = 1 - np.mean(np.power(ret_hat_rpca[ind] - yoos, 2)) / np.mean(np.power(yoos, 2))
print(f"RPCA Total R^2: {r2_rpca[0]:.4f}")
print(f"RPCA Predictive R^2: {r2_rpca[1]:.4f}")
print(f"RPCA Execution Time: {time_rpca:.2f} seconds")

RPCA Total R^2: 0.4431
RPCA Predictive R^2: 0.0375
RPCA Execution Time: 0.10 seconds
```

Part 4: Comparison single run

We compare the performance of IPCA and Regressed-PCA based on three key metrics from a single Monte Carlo simulation run:

- Total R²:** Measures the in-sample explanatory power of the model using contemporaneous factors. It shows how well the model fits the data it was trained on.
- Predictive R²:** Measures the out-of-sample predictive power. It evaluates how well the model forecasts returns on unseen data, using a predictive factor estimate (the expanding window mean of past factors).
- Execution Time:** The wall-clock time required to estimate the model.

The results show that both models achieve very similar explanatory and predictive power, but Regressed-PCA is substantially faster due to its non-iterative estimation procedure.

```
In [23]: results = {
    'Metric': ['Total R^2', 'Predictive R^2', 'Execution Time (s)'],
    'IPCA': [r2_ipca[0], r2_ipca[1], time_ipca],
    'Regressed-PCA': [r2_rpca[0], r2_rpca[1], time_rpca]
}

df_results = pd.DataFrame(results).set_index('Metric')
print("Monte Carlo Summary Table")
print(df_results.round(4))

Monte Carlo Summary Table
              IPCA    Regressed-PCA
Metric
Total R^2          0.4443          0.4431
Predictive R^2      0.0372          0.0375
Execution Time (s)  2.6480          0.1048

Part 5: Monte Carlo 100 simulations

In [25]: # Initialize lists to store results from each simulation
num_simulations = 200
ipca_r2_total = []
ipca_r2_pred = []
ipca_times = []
rpca_r2_total = []
rpca_r2_pred = []
rpca_times = []

for m_sim in range(1, num_simulations + 1):
    if m_sim % 10 == 0:
        print(f"Running simulation {m_sim}/{num_simulations}")
    # Load data for the current simulation
    M, an = 1, m_sim
    P = 50 # number of characteristics
    path = "demo_data"
    data = pd.read_csv(path + "/c_1d.csv" % an, delimiters=",").values
    r = pd.read_csv(path + "/r1d_1d.csv" % (N, an), delimiters=",").values[:, 0]

    # The RPCA and IPCA functions use internal global copies of the data.
    # We must explicitly overwrite them with the newly loaded data for this simulation loop.
    import IPCA
    import RPCA
    IPCA.data = data
    IPCA.r = r
    RPCA.data = data
    RPCA.r = r
    # -----

    # Define panel dimensions and create panel identifiers.
    N = 200
    M = P
    T = 180
    K = 5

    per = np.tile(np.arange(N), T)
    time = np.repeat(np.arange(T), N)
    date = time

    # Split data into train, validation, and out-of-sample sets
    ind = range(0, int(N * T / 3))
    xtrain = data[ind, :]
    ytrain = r[ind]
    trainper = per[ind]
    traindate = time[ind]

    ind = range(int(N * T / 3), int(N * (T * 2 / 3)))
    xtest = data[ind, :]
    ytest = r[ind]
    testper = per[ind]
    testdate = time[ind]

    ind = range(int(N * T * 2 / 3), N * T)
    xoos = data[ind, :]
    yoos = r[ind]
    oosdate = time[ind]
    ooosper = per[ind]

    # Precompute summary statistics
    L = P
    udate = np.unique(time)
    Nt = np.zeros(T)
    for i in range(T):
        ind = time == udate[i]
        Nt[i] = np.sum(ind)
    Nt = Nt.astype(int)

    X = np.zeros((L, T))
    for t in range(T):
        ind = time == udate[t]
        xt = r[ind].reshape(1, Nt[t]).dot(data[ind, :])
        X[:, t] = xt[0, :] / 1.0 / Nt[t]

    W = np.zeros((L, L, T))
    for t in range(T):
        ind = time == udate[t]
        Z = data[ind, :]
        W[:, :, t] = Z.T.dot(Z) / Nt[t]

    # Run IPCA
    start_time_ipca = tm.time()
    ret_hat_ipca, ret_hat_ipca, Gamma_ipca = IPCA.IPCA(X, W, Nt, time, udate)
    end_time_ipca = tm.time()
    time_ipca = end_time_ipca - start_time_ipca

    r2_ipca = np.zeros(2)
    ind = range(int(N * T * 2 / 3), N * T)
    r2_ipca[0] = 1 - np.mean(np.power(ret_hat_ipca[ind] - yoos, 2)) / np.mean(np.power(yoos, 2))
    r2_ipca[1] = 1 - np.mean(np.power(ret_hat_ipca[ind] - yoos, 2)) / np.mean(np.power(yoos, 2))
    ipca_r2_total.append(r2_ipca[0])
    ipca_r2_pred.append(r2_ipca[1])
    ipca_times.append(time_ipca)

    # Run RPCA
    start_time_rpca = tm.time()
    ret_hat_rpca, ret_hat_rpca, Gamma_rpca = RPCA.RegressedPCA(X, W, Nt, time, udate)
    end_time_rpca = tm.time()
    time_rpca = end_time_rpca - start_time_rpca

    r2_rpca = np.zeros(2)
    ind = range(int(N * T * 2 / 3), N * T)
    r2_rpca[0] = 1 - np.mean(np.power(ret_hat_rpca[ind] - yoos, 2)) / np.mean(np.power(yoos, 2))
    r2_rpca[1] = 1 - np.mean(np.power(ret_hat_rpca[ind] - yoos, 2)) / np.mean(np.power(yoos, 2))
    rpca_r2_total.append(r2_rpca[0])
    rpca_r2_pred.append(r2_rpca[1])
    rpca_times.append(time_rpca)

Running simulation 10/200
Running simulation 20/200
Running simulation 30/200
Running simulation 40/200
Running simulation 50/200
Running simulation 60/200
Running simulation 70/200
Running simulation 80/200
Running simulation 90/200
Running simulation 100/200
Running simulation 110/200
Running simulation 120/200
Running simulation 130/200
Running simulation 140/200
Running simulation 150/200
Running simulation 160/200
Running simulation 170/200
Running simulation 180/200
Running simulation 190/200
Running simulation 200/200

In [26]: # Calculate average results
avg_ipca_r2_total = np.mean(ipca_r2_total)
avg_ipca_r2_pred = np.mean(ipca_r2_pred)
avg_ipca_time = np.mean(ipca_times)

avg_rpca_r2_total = np.mean(rpca_r2_total)
avg_rpca_r2_pred = np.mean(rpca_r2_pred)
avg_rpca_time = np.mean(rpca_times)

# Calculate standard deviations
std_ipca_r2_total = np.std(ipca_r2_total)
std_ipca_r2_pred = np.std(ipca_r2_pred)
std_ipca_time = np.std(ipca_times)

std_rpca_r2_total = np.std(rpca_r2_total)
std_rpca_r2_pred = np.std(rpca_r2_pred)
std_rpca_time = np.std(rpca_times)

# Create summary table
results = {
    'Metric': ['Total R^2', 'Predictive R^2', 'Execution Time (s)'],
    'IPCA': [
        f'{avg_ipca_r2_total:.4f} ± {std_ipca_r2_total:.4f}',
        f'{avg_ipca_r2_pred:.4f} ± {std_ipca_r2_pred:.4f}',
        f'{avg_ipca_time:.4f} ± {std_ipca_time:.4f}'
    ],
    'Regressed-PCA': [
        f'{avg_rpca_r2_total:.4f} ± {std_rpca_r2_total:.4f}',
        f'{avg_rpca_r2_pred:.4f} ± {std_rpca_r2_pred:.4f}',
        f'{avg_rpca_time:.4f} ± {std_rpca_time:.4f}'
    ]
}

df_results = pd.DataFrame(results).set_index('Metric')
print(f"Monte Carlo Summary Table (Averages over {num_simulations} simulations)")
print(df_results)

Monte Carlo Summary Table (Averages over 200 simulations)
              IPCA    Regressed-PCA
Metric
Total R^2          0.4121 ± 0.0288    0.4112 ± 0.0289
Predictive R^2      0.0393 ± 0.0291    0.0393 ± 0.0291
Execution Time (s)  4.2552 ± 4.4666    0.0635 ± 0.0036
```

```
In [29]: fig, axes = plt.subplots(3, 2, figsize=(12, 12))
fig.suptitle('Distribution of Results over 200 Simulations', fontsize=16)
axes[0, 0].hist(ipca_r2_total, bins=20, alpha=0.7, label='IPCA')
axes[0, 0].set_title('IPCA: Total R^2')
axes[0, 0].legend()
axes[0, 1].hist(rpca_r2_total, bins=20, alpha=0.7, label='RPCA', color='orange')
axes[0, 1].set_title('RPCA: Total R^2')
axes[0, 1].legend()
axes[1, 0].hist(ipca_r2_pred, bins=20, alpha=0.7, label='IPCA')
axes[1, 0].set_title('IPCA: Predictive R^2')
axes[1, 0].legend()
axes[1, 1].hist(rpca_r2_pred, bins=20, alpha=0.7, label='RPCA', color='orange')
axes[1, 1].set_title('RPCA: Predictive R^2')
axes[1, 1].legend()
axes[2, 0].hist(ipca_times, bins=20, alpha=0.7, label='IPCA')
axes[2, 0].set_title('IPCA: Execution Time (s)')
axes[2, 0].legend()
axes[2, 1].hist(rpca_times, bins=20, alpha=0.7, label='RPCA', color='orange')
axes[2, 1].set_title('RPCA: Execution Time (s)')
axes[2, 1].legend()
plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()
```

Distribution of Results over 200 Simulations

